UNIVERSIDADE KIMPA VITA

INSTITUTO POLITÉCNICO

LICENCIATURA EM ENGENHARIA INFORMÁTICA

TRABALHO PRÁTICO DE SGBD II

MODELAGEM E IMPLEMENTAÇÃO DE UM SISTEMA DE GESTÃO DE EVENTOS ACADÉMICOS (WORKSHOPS)

III ANO

PERÍODO: LABORAL

ORIENTADO POR:

Moyo Kanivengidio MSc.

Uíge, 2025-2026

EQUIPA DESENVOLVEDORA

O trabalho foi desenvolvido em grupo. A tabela seguinte apresenta os membros da equipa e as respectivas contribuições ao projecto:

Nome	Responsabilidade	Descrição da Contribuição
Nsumbo António Ngombo	Modelagem Conceptual e Lógica	MER e o Modelo Lógico, definindo entidades, atributos, cardinalidades e relacionamentos.
Clemente Manuel D. José	Modelo Físico (SQL)	Scripts DDL e DML e as consultas SQL.
Arnano Quiesse Mavinga	Levantamento de Requisitos	Identificação as necessidades do negócio e as regras de integridade.
Augusto Paulo	Levantamento de Requisitos	Identificação as necessidades do negócio e as regras de integridade.
João Nkembo	Testes e Validação	Validação as consultas SQL e verificou a conformidade com os requisitos.

INTRODUÇÃO

O presente trabalho foi desenvolvido no âmbito da disciplina de Sistemas de Gestão de Bases de Dados II (SGBD), leccionada no Instituto Politécnico da Universidade Kimpa Vita do Uíge. O objectivo principal consiste em projectar, implementar e popular uma base de dados relacional completa para suportar a gestão de eventos académicos do tipo Workshop.

Na sociedade contemporânea, caracterizada pela crescente digitalização dos processos organizacionais, a informação tornou-se um recurso fundamental para a gestão eficiente das instituições. Neste enquadramento, as bases de dados surgem como solução estruturada para organizar e gerir informações de forma eficiente, garantindo que os dados sejam armazenados de forma consistente, segura e acessível.

Um Workshop é um evento académico onde vários oradores apresentam temas em diferentes sessões, e os participantes podem inscrever-se para assistir. O sistema desenvolvido permite controlar de forma integrada todos os aspectos do evento: a sua organização departamental, as sessões, os oradores que apresentam, os participantes e o processo de inscrição.

O trabalho percorreu as seguintes fases:

- ✓ Análise dos requisitos do Sistema;
- ✓ Criação do Modelo Entidade-Relação (modelo conceptual);
- ✓ Conversão para o modelo lógico com normalização até à 3FN;
- ✓ Implementação física com scripts SQL (DDL e DML);
- ✓ Elaboração de consultas SQL para relatórios.

1. ANÁLISE DE REQUISITOS

Antes de iniciar a modelagem, foi realizada uma análise detalhada dos requisitos do sistema, identificando as entidades, os atributos, as relações e as restrições de integridade necessárias.

1.1 Entidades Identificadas

As principais entidades do sistema são:

Entidade	Descrição
DEPARTAMENTO	Departamento académico responsável pela organização dos eventos.
EVENTO	O Workshop em si – contém informações gerais do evento.
SESSAO	Apresentação específica dentro de um evento, com hora, local e duração.
ORADOR	Especialista convidado a apresentar numa ou mais sessões.
PARTICIPANTE	Pessoa que se inscreve para assistir ao evento.
INSCRICAO	Registo formal da inscrição de um participante, processado por um funcionário.
FUNCIONARIO	Colaborador administrativo responsável por processar inscrições.
EVENTO_INSCRICAO	Entidade associativa que liga cada inscrição ao evento correspondente.
ORADORES_SESSAO	Entidade associativa que liga oradores às sessões em que apresentam.

1.2 Regras de Negócio (Cardinalidades)

- **Departamento → Eventos (1:N):** Um departamento pode organizar vários eventos; cada evento pertence a um só departamento.

- **Evento → Sessões (1:N):** Um evento tem várias sessões; cada sessão pertence a um só evento.
- **Sessão ↔ Orador (N:M):** Uma sessão pode ter vários oradores e um orador pode apresentar em várias sessões (resolvido via ORADORES_SESSAO).
- **Participante → Inscrições (1:N):** Um participante pode ter várias inscrições; cada inscrição pertence a um só participante.
- **Inscrição → Evento (via EVENTO_INSCRICAO N:M):** Uma inscrição está associada a um evento; um evento pode ter várias inscrições.
- **Funcionário → Inscrições (1:N):** Um funcionário pode processar várias inscrições.

1.3 Restrições de Integridade Identificadas

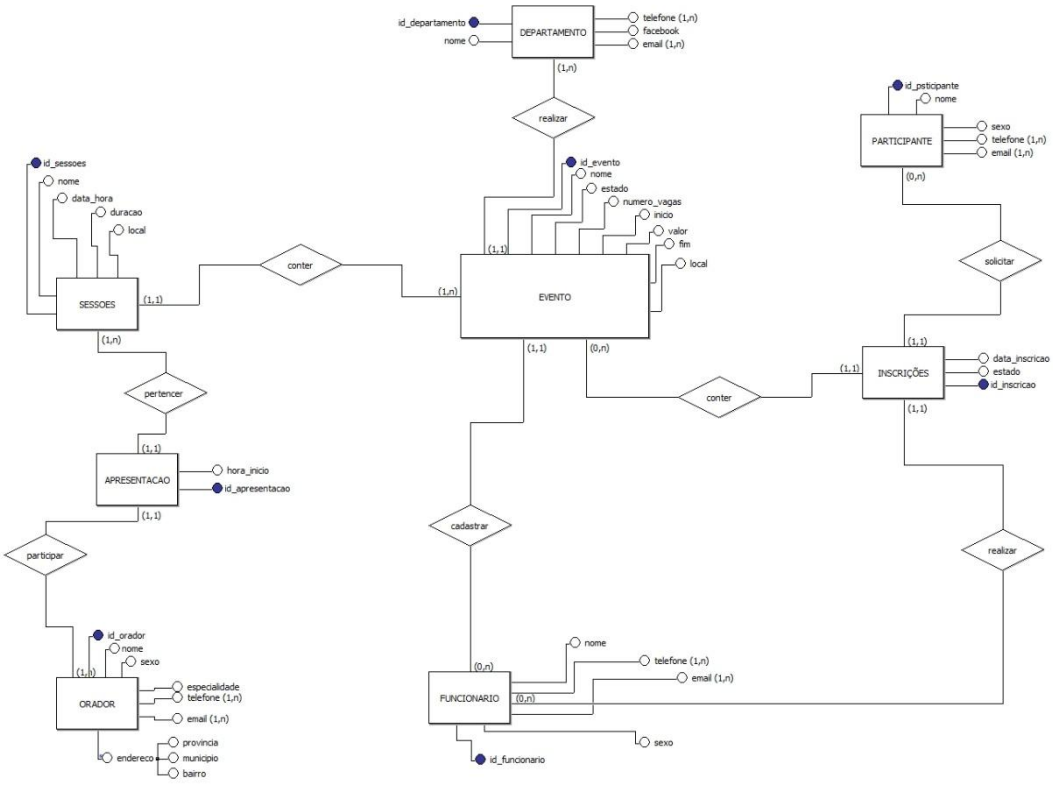
Com base nos princípios descritos foram identificadas as seguintes restrições:

Restrição	Tipo	Implementação SQL
DataFim (Fim) deve ser posterior a DataInicio (Inicio)	CHECK – Domínio	CHECK (Fim > Inicio)
Capacidade (Numero_vaga) deve ser número positivo	CHECK – Domínio	CHECK (Numero_vaga > 0)
Preço da inscrição deve ser >= 0	CHECK – Domínio	CHECK (Preco >= 0)
Chave primária de cada tabela – NOT NULL e UNIQUE	Integridade de Entidade	PRIMARY KEY AUTO_INCREMENT
FK em Inscricao referencia Participante existente	Integridade Referencial	FOREIGN KEY ... REFERENCES
FK em SESSAO referencia Evento existente	Integridade Referencial	FOREIGN KEY ... REFERENCES

Restrição	Tipo	Implementação SQL
Estado da inscrição deve ser valor válido	CHECK – Domínio	CHECK (Estado IN ('Confirmada','Pendente','Cancelada'))
Email dos contactos deve ser único por entidade	Unicidade	UNIQUE (Email)
Valores da inscrição devem ser > 0	CHECK – Domínio	CHECK (Valores > 0)

2. MODELO CONCEPTUAL

2.1 Diagrama do Modelo Conceptual



2.2 Descrição dos Atributos Principais

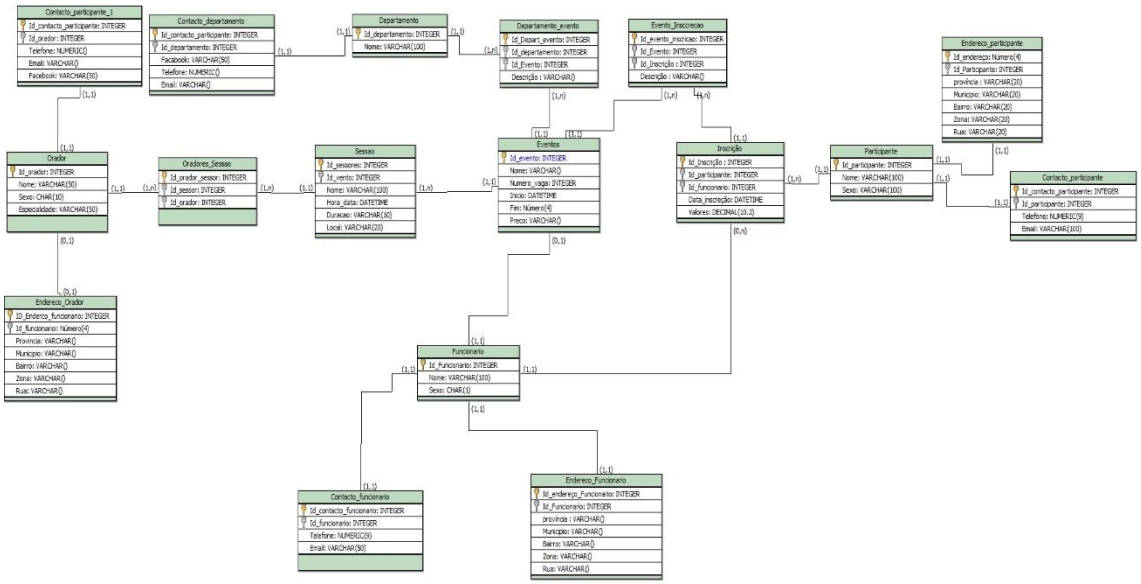
A tabela seguinte resume os principais atributos de cada entidade identificada no MER:

Entidade	Atributos	Chave Primária
DEPARTAMENTO	Id_departamento, Nome	Id_departamento
EVENTO	Id_evento, Nome, Numero_vaga, Inicio, Fim, Preco	Id_evento
SESSAO	Id_sessao, Nome, _Local, Duracao, Hora_data	Id_sessao
ORADOR	Id_orador, Nome, Sexo, Especialidade	Id_orador
PARTICIPANTE	Id_participante, Nome, Sexo	Id_participante
INSCRICAO	Id_Inscricao, Data_inscreção, Valores, Estado	Id_Inscricao
FUNCIONARIO	Id_Funcionario, Nome, Sexo	Id_Funcionario
EVENTO_INSCRICAO	Id_evento_Inscricao, Id_Inscricao (FK), Id_evento (FK)	Id_evento_Inscricao
ORADORES_SESSAO	Id_orador_sessao, Id_sessao (FK), Id_orador (FK)	Id_orador_sessao

3. MODELO LÓGICO E NORMALIZAÇÃO

O modelo lógico foi gerado a partir do modelo conceptual. Nesta fase, cada entidade é convertida numa tabela e as relações são representadas por chaves estrangeiras, seguindo as regras de mapeamento MER→Relacional.

3.1 Diagrama do Modelo Lógico



4. MODELO FÍSICO E IMPLEMENTAÇÃO SQL

A implementação física foi realizada em MySQL, conforme exigido pelo enunciado. Os scripts incluem todas as restrições de integridade identificadas, implementadas via comandos DDL.

4.1 Criação da Base de Dados e Tabelas (DDL)

Os comandos DDL seguintes criam a base de dados e todas as tabelas com as respectivas restrições de integridade (PRIMARY KEY, FOREIGN KEY, NOT NULL, CHECK):

```
-- Criação de base de dados
CREATE DATABASE BD_EVENTO_ACADEMICO ;
USE BD_EVENTO_ACADEMICO ;

CREATE TABLE Funcionario (
Id_Funcionario INTEGER PRIMARY KEY auto_increment ,
Nome VARCHAR(100),
Sexo CHAR(1)
);
CREATE TABLE Endereco_Funcionario (
Id_endereço_Funcionario integer primary key auto_increment,
Id_Funcionario INTEGER,
província  VARCHAR(20),
```

```
Municipio VARCHAR(20),
Bairro VARCHAR(20),
Zona VARCHAR(20),
Rua VARCHAR(20),
foreign key (Id_Funcionario) references Funcionario(Id_Funcionario)
);
CREATE TABLE Contacto_funcionario (
Id_contacto_funcionario integer primary key auto_increment,
Id_funcionario INTEGER,
Telefone NUMERIC(9),
Email VARCHAR(50),
FOREIGN KEY(Id_funcionario) REFERENCES Funcionario (Id_funcionario)
);

CREATE TABLE Participante (
Id_participante INTEGER PRIMARY KEY auto_increment,
Nome VARCHAR(100),
Sexo VARCHAR(100)
);
CREATE TABLE Contacto_participante (
Id_contacto_participante integer primary key auto_increment,
Id_participante INTEGER,
Telefone NUMERIC(9),
Email VARCHAR(100),
FOREIGN KEY(Id_participante) REFERENCES Participante (Id_participante)
);
CREATE TABLE  Endereco_participante (
Id_endereço integer primary key auto_increment,
Id_Participante INTEGER,
província  VARCHAR(20),
Municipio VARCHAR(20),
Bairro VARCHAR(20),
Zona VARCHAR(20),
Rua VARCHAR(20),
FOREIGN KEY(Id_Participante) REFERENCES Participante (Id_participante)
```

```
);
show tables;
CREATE TABLE Orador (
Id_orador INTEGER PRIMARY KEY AUTO_INCREMENT,
Nome VARCHAR(50),
Sexo CHAR(10),
Especialidade VARCHAR(50)
);
CREATE TABLE Oradores_Sessao (
Id_orador_sessao INTEGER PRIMARY KEY AUTO_INCREMENT,
Id_sessao INTEGER,
Id_orador INTEGER,
FOREIGN KEY(Id_sessao) REFERENCES Sessao (Id_Sessao),
FOREIGN KEY(Id_orador) REFERENCES Orador (Id_orador)
);
select *from Oradores_Sessao;
CREATE TABLE Endereco_Orador (
ID_Enderco_funcionario INTEGER PRIMARY KEY AUTO_INCREMENT,
Id_orador INTEGER,
Provincia VARCHAR(50),
Municipio VARCHAR(50),
Bairro VARCHAR(50),
Zona VARCHAR(50),
Rua VARCHAR(50),
FOREIGN KEY(Id_orador) REFERENCES Orador (Id_orador)
);
CREATE TABLE Contacto_orador (
Id_contacto_orador integer primary key auto_increment,
Id_orador INTEGER,
Telefone NUMERIC(9),
Email VARCHAR(50),
Facbook varchar(20),
FOREIGN KEY(Id_orador) REFERENCES Orador (Id_orador)
);
select *from Inscricao;
```

```
show tables;
CREATE TABLE  Inscricao(
Id_Inscricao  INTEGER PRIMARY KEY auto_increment,
Id_participante INTEGER,
Id_funcionario INTEGER,
Data_inscreção DATETIME,
Valores DECIMAL(10,2),
Estado VARCHAR(50),
FOREIGN KEY(Id_participante) REFERENCES Participante (Id_participante),
FOREIGN KEY(Id_funcionario) REFERENCES Funcionario (Id_Funcionario)
);
CREATE TABLE Evento_Inscrecao (
Id_evento_Inscricao INTEGER PRIMARY KEY AUTO_INCREMENT,
Id_Inscricao INTEGER,
Id_evento INTEGER,
FOREIGN KEY(Id_Inscricao) REFERENCES Inscricao (Id_Inscricao),
FOREIGN KEY(Id_evento) REFERENCES Eventos (Id_evento)
);
show tables;
CREATE TABLE SESSAO (
Id_sessao INTEGER PRIMARY KEY AUTO_INCREMENT,
Id_vento INTEGER,
Nome VARCHAR(100),
_Local VARCHAR(20),
Duracao date,
Hora_data DATETIME,
foreign key (Id_vento) references Eventos(Id_evento)
);
CREATE TABLE Eventos (
Id_evento INTEGER PRIMARY KEY auto_increment,
Nome VARCHAR(100),
Numero_vaga INTEGER,
Inicio DATETIME,
Fim datetime,
Preco decimal(10,2)
```

```
);
show tables;
CREATE TABLE Departamento (
Id_departamento INTEGER PRIMARY KEY auto_increment,
Nome VARCHAR(100)
);
CREATE TABLE departamento_evento (
Id_dapartamento_evento INTEGER PRIMARY KEY AUTO_INCREMENT,
Id_departamento INTEGER,
Id_evento INTEGER,
FOREIGN KEY(Id_departamento) REFERENCES Departamento(Id_departamento),
FOREIGN KEY(Id_evento) REFERENCES Eventos (Id_evento)
);
show tables;
CREATE TABLE Contacto_departamento (
Id_contacto_participante INTEGER PRIMARY KEY auto_increment,
Id_departamento INTEGER,
Telefone NUMERIC(9),
Email VARCHAR(100),
FOREIGN KEY(Id_departamento) REFERENCES Departamento (Id_departamento)
);
```

4.2 Inserção de Dados (DML – INSERT INTO)

Os comandos DML populam a base de dados com 5 eventos, 10 oradores, 20 participantes e 30 inscrições, com dados realistas do contexto angolano. O script completo encontra-se no ficheiro DML_Eventos_Academicos.sql, disponível no repositório GitHub do projecto.

A lógica de inserção respeita a seguinte ordem para garantir a integridade referencial:

- Departamento → Eventos → departamento_evento
- SESSAO (depende de Eventos)
- Orador → Contacto_orador → Endereco_Orador → Oradores_Sessao
- Participante → Contacto_participante → Endereco_participante
- Funcionario → Contacto_funcionario → Endereco_Funcionario

- Inscricao → Evento_Inscrecao

Decisão técnica adoptada: foram criados 30 registos em Inscricao (cada um com participante, funcionário, data, valor e estado) e 30 registos correspondentes em Evento_Inscrecao, garantindo volume real e consistência referencial. A distribuição inclui participantes inscritos em mais de um evento e estados variados (Confirmada, Pendente, Cancelada), o que torna as consultas SQL mais ricas e representativas.

5. CONSULTA E RELATÓRIOS SQL

As cinco consultas a seguir implementam os relatórios exigidos pelo enunciado, utilizando JOIN, GROUP BY, HAVING e subconsultas.

Consulta 1 – Eventos com Oradores e Total de Participantes

Lista todos os eventos com os respectivos oradores e o número total de participantes inscritos com estado 'Confirmada'.

Consulta 2 – Participantes Inscritos em Mais de um Evento

Identifica os participantes que se inscreveram em pelo menos dois eventos distintos, independentemente do estado da inscrição.

Consulta 3 – Sessões de um Evento Específico Ordenadas por Data e Hora

Obtém a lista de sessões do evento com Id_evento = 1 (Workshop de IA), ordenadas cronologicamente.

Consulta 4 – Receita Total Gerada por Cada Evento

Calcula a receita total por evento, considerando apenas as inscrições com estado 'Confirmada'.

Consulta 5 – Oradores em Eventos de Mais de uma Cidade (Província)

Identifica os oradores que apresentam em sessões de eventos associados a mais de uma província distinta, com base no endereço do orador cruzado com a localização dos eventos (representada pela província do evento via sessão).

6. DESAFIOS ENCONTRADOS

Durante o desenvolvimento do projecto, o grupo enfrentou os seguintes desafios:

- Definição correcta das cardinalidades: No início houve dificuldades em estabelecer em que lado da relação devia ficar cada cardinalidade. Após revisão dos princípios apresentados por Kanivengidio (2026) – nomeadamente as relações 1:1, 1:N e N:M [Cap. 6, pp. 63-65] –, compreendeu-se que a cardinalidade junto a uma entidade descreve quantas instâncias dessa entidade participam na relação.
- Atributos multivalorados: Perceber que atributos como Telefone e Email, por poderem ter vários valores, precisavam de tabelas separadas no modelo lógico. Esta abordagem está alinhada com o princípio da 1FN e com a regra de mapeamento de atributos multivalorados descrita por Kanivengidio (2026) [Cap. 6, p. 65].
- Relação N:M entre Sessão e Orador: A relação muitos-para-muitos exigiu a criação da entidade associativa ORADORES_SESSAO para resolver a relação, seguindo a regra de mapeamento de relações N:M para o modelo relacional [Cap. 6, p. 66].
- Ordem de criação das tabelas (DDL): As dependências de chaves estrangeiras exigiram uma ordem específica de criação das tabelas. Por exemplo, SESSAO só pode ser criada após Eventos, pois referencia a sua chave primária.
- Volume e coerência dos dados (DML): Garantir que 30 inscrições estavam correctamente ligadas aos 5 eventos via Evento_Inscrecao, mantendo a integridade referencial e incluindo participantes com múltiplas inscrições para que as consultas SELECT devolvessem resultados significativos.

CONCLUSÃO

Este trabalho permitiu ao grupo aplicar de forma prática os conceitos teóricos leccionados na disciplina de SGBD. O processo completo – desde a análise de requisitos até à implementação física e às consultas SQL – demonstrou a importância de um bom design inicial para garantir que o sistema funcione correctamente, sem dados repetidos ou inconsistentes.

Como afirma Kanivengidio (2026), "um bom design é crucial para o desempenho, a integridade e a manutenibilidade de qualquer sistema de informação" [Cap. 6, p. 62]. O sistema desenvolvido concretiza este princípio, gerindo de forma organizada todos os aspectos de um Workshop académico: eventos, sessões, oradores, participantes e inscrições.